01 / U2 SQL Server - Agregaciones

Contar, sumar y resumir los datos del club

Material de estudio para Eric Escobar
Viernes 30 de octubre de 2026 · C#/.NET

Plan original: Cronograma_CSharp_NET.pdf, página 2, Semana 2, fila del viernes 30/10. Tema: COUNT, SUM, GROUP BY y HAVING. Actividad: calcular ocupación, reservas y cupos por clase. Evidencia esperada: cinco indicadores correctos en SQL.

Lo que deberías poder hacer al terminar

- Elegir entre contar filas y sumar cantidades, explicando la diferencia.
- Resumir reservas por clase y conservar las clases sin reservas.
- Aplicar un filtro a los grupos con HAVING y distinguirlo de WHERE.
- Calcular ocupación y lugares libres sin duplicar capacidades ni perder decimales.
- Contrastar cada resultado con las filas de origen y explicar qué representa.

Continuidad confirmada

JOIN parte 2 quedó completo el 29/10: seis actividades revisadas. Conservamos ese avance. Hoy iniciamos agregaciones desde cero; ningún indicador de esta sesión se considera resuelto todavía.

Herramienta: SQL Server Management Studio (SSMS). Base de práctica: ClubPadel. Vamos a trabajar una actividad por vez y a revisar consulta, ejecución y razonamiento.

02 / Alcance, tiempos y progreso

El cronograma fija cuatro horas y cinco indicadores. Para acompañarte, los organizamos en estas cinco actividades; sus códigos y tiempos son una distribución didáctica del bloque, no una enumeración literal del plan.

Código	Resultado a construir	Min.	Estado
A01	Cantidad total de reservas con COUNT	25	En curso
A02	Capacidad total del club con SUM	20	Pendiente
A03	Reservas por clase, incluidas las clases con cero	40	Pendiente
A04	Porcentaje de ocupación por clase	25	Pendiente
A05	Lugares libres por clase e informe filtrado con HAVING	45	Pendiente

Bloque del plan	Tiempo
Repaso de JOIN y lectura de fuentes	20 min
Teoría en bloques pequeños	45 min
Ejercicios A01 a A04	110 min
Proyecto A05	45 min
Corrección y registro de cierre	20 min

Estado inicial: 0 % · 0 de 5 actividades completadas. A01 está en curso. Una actividad suma únicamente después de revisar el código, su resultado y tu explicación. Ver una solución no completa la práctica.

Progreso = actividades completadas ÷ 5 × 100. Cada actividad completa aporta 20 puntos porcentuales. El total se mantiene en cinco salvo acuerdo de cambio de alcance.

Pausas fuera del cómputo. Se respeta tu ritmo; si falta tiempo, queda registrado el punto pendiente y se reprograma. No se agrega una quinta hora autónoma.

03 / Datos de partida y significado

Estas son las últimas filas confirmadas en ClubPadel al cerrar JOIN. Son la referencia para comprobar resultados; si tu base cambió, contrastamos los datos antes de evaluar una consulta.

IdAlumno	Nombre	Edad
1	Eric	29
2	Carlos	34
3	Leo	15
4	Maria	30
5	Cesar	45
6	Ana	23
7	Julia	26

IdClase	Nombre	Cupo
1	Septima	8
2	Sexta	8
3	Quinta	6
4	Cuarta	4
5	Tercera	4
6	Octava	8

IdReserva	IdAlumno	IdClase
1	3	2
2	4	1
3	1	3
4	5	4
5	2	5
6	6	1

Julia no tiene reservas y Octava no tiene reservas asociadas. IdReserva es la clave primaria de Reserva. Los identificadores de alumno y clase en Reserva son claves foráneas y pueden repetirse.

04 / Qué es agregar y cómo leer COUNT

Una **fila** es un registro. Una **función** recibe información y devuelve un resultado. Una **función de agregación** resume un conjunto de valores: COUNT cuenta; SUM suma. Sin agrupación, el resumen suele ocupar una sola fila.

Ejemplo resuelto: contar alumnos

```
USE ClubPadel;

SELECT COUNT(*) AS TotalAlumnos
FROM Alumno;
```

TotalAlumnos
7

• USE ClubPadel establece la base donde se ejecutan las consultas.

• SELECT indica qué resultado queremos mostrar.

• COUNT(*) cuenta todas las filas de la tabla indicada por FROM.

• AS TotalAlumnos da un nombre legible a la columna calculada; no agrega una columna a la tabla.

• FROM Alumno establece la fuente. El punto y coma termina la instrucción.

El resultado tiene una fila que dice 7. Eso no significa que exista un solo alumno: significa que resumimos siete registros en un total. SELECT * mostraría sus datos individuales; SELECT COUNT(*) muestra cuántos registros hay.

Contar no es sumar identificadores

Los identificadores son etiquetas, no cantidades de alumnos. SUM(IdAlumno) sumaría sus números y no respondería cuántas personas hay. Tampoco alcanza con mirar el identificador más alto: pueden existir huecos por bajas.

Práctica A01: contar reservas será tu ejercicio. Este ejemplo cuenta alumnos; no es la solución de A01. Ejecutá solamente la instrucción que estemos trabajando en SSMS.

05 / Ejemplos separados de tu práctica

Desde esta página, los ejemplos resueltos usan tres clases ficticias: Basico, Intermedio y Avanzado. No son las clases de ClubPadel. Así podés estudiar el procedimiento sin recibir las soluciones completas de A02 a A05.

IdClase	Nombre	Cupo	Reservas asociadas
101	Basico	4	201 y 202
102	Intermedio	6	203
103	Avanzado	5	Ninguna

Si querés ejecutar los ejemplos por tu cuenta, este bloque crea dos **tablas temporales**. El prefijo # las identifica: existen en la conexión actual y se eliminan al cerrarla. Preparalas una sola vez en una pestaña nueva de SSMS y usá esa misma pestaña para los ejemplos. No insertan datos en Alumno, Clase ni Reserva.

```
CREATE TABLE #ClaseDemo
(
    IdClase INT PRIMARY KEY,
    Nombre NVARCHAR(50) NOT NULL,
    Cupo INT NOT NULL
);

CREATE TABLE #ReservaDemo
(
    IdReserva INT PRIMARY KEY,
    IdClase INT NOT NULL
);

INSERT INTO #ClaseDemo (IdClase, Nombre, Cupo)
VALUES (101, N'Basico', 4),
       (102, N'Intermedio', 6),
       (103, N'Avanzado', 5);

INSERT INTO #ReservaDemo (IdReserva, IdClase)
VALUES (201, 101), (202, 101), (203, 102);
```

La primera tabla guarda clases y capacidad. La segunda guarda qué clase corresponde a cada reserva. Las columnas bastan para estos ejemplos. La preparación es una referencia opcional, no una actividad adicional de la barra. Si ya existen las tablas # en esa conexión, no vuelvas a ejecutar sus CREATE e INSERT.

06 / SUM: sumar cantidades

SUM(columna) suma los valores numéricos de una columna. Se usa cuando cada fila aporta una cantidad: capacidad, importe o unidades. En cambio, COUNT responde cuántas filas o valores no nulos hay.

```
SELECT SUM(Cupo) AS CapacidadTotal
FROM #ClaseDemo;
```

CapacidadTotal
15

SUM(Cupo) toma 4, 6 y 5. Los suma: 4 + 6 + 5 = 15 lugares de capacidad. AS nombra ese resumen. FROM elige las tres clases del ejemplo. No hace falta unir reservas para sumar una capacidad almacenada una vez por clase.

Dos preguntas, dos cálculos

```
SELECT COUNT(*) AS CantidadClases,
       SUM(Cupo) AS CapacidadTotal
FROM #ClaseDemo;
```

CantidadClases	CapacidadTotal
3	15

Hay tres clases y capacidad para quince lugares. No son dos formas de medir lo mismo: el primer número cuenta registros y el segundo suma sus cantidades.

WHERE antes del resumen

```
SELECT SUM(Cupo) AS CapacidadSeleccionada
FROM #ClaseDemo
WHERE Cupo >= 5;
```

Primero quedan Intermedio (6) y Avanzado (5); después SUM devuelve 11. El filtro selecciona filas antes de sumarlas.

SUM ignora valores NULL; no los transforma en cero. Si no tiene valores no nulos para sumar, devuelve NULL. En nuestras clases Cupo está definido como NOT NULL. COUNT sin GROUP BY devuelve cero cuando no hay filas que contar.

07 / GROUP BY: un resumen por grupo

Un **grupo** reúne filas con los mismos valores en las columnas elegidas. GROUP BY permite obtener, por ejemplo, una cantidad de reservas para cada IdClase, en lugar de un solo total de todas las reservas.

```
SELECT IdClase,
       COUNT(*) AS CantidadReservas
FROM #ReservaDemo
GROUP BY IdClase
ORDER BY IdClase ASC;
```

IdClase	CantidadReservas
101	2
102	1

- FROM toma las tres reservas del ejemplo.
- GROUP BY IdClase reúne 201 y 202 en el grupo 101; la reserva 203 queda en el grupo 102.
- COUNT(*) se calcula dentro de cada grupo: dos y una.
- SELECT muestra la clave del grupo y el resumen.
- ORDER BY ordena el resultado. GROUP BY no garantiza un orden.

Por qué no aparece Avanzado

La fuente es Reserva: no existe allí una fila con IdClase 103. Un grupo no se inventa por la existencia de una clase en otra tabla. Para incluir clases sin reservas comenzaremos desde Clase y usaremos LEFT JOIN.

Columnas que deben agruparse

En estas consultas simples, las columnas que mostrás sin una agregación deben estar en GROUP BY. Si seleccionás IdClase y Nombre junto con COUNT, agrupá por IdClase y Nombre. Si además mostrás Cupo, incluilo también. Agrupar por identificador evita mezclar clases diferentes que compartieran nombre.

08 / Contar reservas con LEFT JOIN

Queremos una fila por clase, incluso cuando no tenga reservas. La tabla izquierda debe ser Clase. El ON relaciona la clase de la reserva con la clase que estamos listando.

```
SELECT c.IdClase,
       c.Nombre AS Clase,
       COUNT(r.IdReserva) AS CantidadReservas
FROM #ClaseDemo AS c
LEFT JOIN #ReservaDemo AS r
    ON c.IdClase = r.IdClase
GROUP BY c.IdClase, c.Nombre
ORDER BY c.IdClase ASC;
```

IdClase	Clase	CantidadReservas
101	Basico	2
102	Intermedio	1
103	Avanzado	0

• SELECT elige la identificación, el nombre y el conteo por grupo.

• FROM #ClaseDemo AS c establece qué filas queremos conservar.

• LEFT JOIN agrega reservas cuando encuentra coincidencias; conserva Avanzado aunque no tenga ninguna.

• ON compara dos IdClase relacionados. No compara IdAlumno con IdClase.

• COUNT(r.IdReserva) cuenta identificadores de reserva no NULL.

• GROUP BY reúne las filas por clase; ORDER BY presenta sus identificadores de menor a mayor.

En Avanzado: el LEFT JOIN produce una fila con los datos de la clase y NULL del lado de Reserva. COUNT(r.IdReserva) no cuenta ese NULL, así que devuelve 0. No se creó una reserva nueva.

09 / La trampa de contar o sumar tras unir

COUNT(*) no equivale siempre a contar reservas

Clase	Filas del LEFT JOIN	COUNT(*)	COUNT(r.IdReserva)
Basico	2	2	2
Intermedio	1	1	1
Avanzado	1	1	0

COUNT(*) cuenta filas del resultado, incluida la fila que conserva Avanzado. COUNT(r.IdReserva) cuenta solamente valores no NULL de esa columna. Como IdReserva es clave primaria, identifica reservas reales cuando existe una coincidencia.

Por eso, para reservas por clase con LEFT JOIN elegimos COUNT del identificador de Reserva. Contar c.IdClase tampoco serviría para detectar cero reservas: el identificador de la clase sí está presente en la fila conservada.

La capacidad puede repetirse en el resultado

Clase	Cupo mostrado tras la unión
Basico - reserva 201	4
Basico - reserva 202	4
Intermedio - reserva 203	6
Avanzado - sin reserva	5

Sumar Cupo sobre esas cuatro filas daría 19, pero la capacidad real del ejemplo es 15. Se contó dos veces la misma capacidad de Basico. No cambió la tabla: cambió cuántas veces aparece cada clase en el resultado.

Para capacidad total, sumá sobre Clase antes de mezclarla con reservas. Para un resumen por clase, mostrás c.Cupo una sola vez por grupo. DISTINCT conserva valores distintos: SUM(DISTINCT Cupo) tampoco es una solución general, porque dos clases con igual capacidad deben aportar ambas.

10 / HAVING y WHERE: filtros distintos

WHERE selecciona filas antes de agrupar. **HAVING** selecciona grupos según una condición; puede usar el conteo o la suma calculada para cada grupo. Usalo cuando la pregunta depende del resumen.

Ejemplo: clases con al menos dos reservas

```
SELECT c.IdClase, c.Nombre AS Clase,
       COUNT(r.IdReserva) AS CantidadReservas
FROM #ClaseDemo AS c
LEFT JOIN #ReservaDemo AS r
    ON c.IdClase = r.IdClase
GROUP BY c.IdClase, c.Nombre
HAVING COUNT(r.IdReserva) >= 2
ORDER BY c.IdClase ASC;
```

IdClase	Clase	CantidadReservas
101	Basico	2

Primero se relacionan clases y reservas. Se forma un grupo por clase y se calcula su conteo. HAVING conserva el grupo Basico porque tiene dos reservas; descarta los otros dos grupos. ORDER BY ordena lo que queda.

Pregunta	Cláusula adecuada
¿Qué clases tienen Cupo de al menos 5?	WHERE c.Cupo >= 5
¿Qué clases reúnen dos o más reservas?	HAVING COUNT(r.IdReserva) >= 2

No escribas WHERE COUNT(...) >= 2: el filtro de filas no evalúa ese resumen por grupo. En HAVING repetimos la expresión agregada; el alias de SELECT no está disponible allí en este patrón de SQL Server.

Orden de escritura habitual: SELECT, FROM/JOIN/ON, WHERE si hace falta, GROUP BY, HAVING si hace falta y ORDER BY. No todas las cláusulas son obligatorias.

11 / Ocupación: una razón con decimales

Un **indicador** es una medida que responde una pregunta concreta. KPI es la sigla inglesa de indicador clave de desempeño. En esta práctica, ocupación por clase significa reservas registradas divididas por la capacidad de esa clase, multiplicado por 100.

Supuesto del ejercicio: cada fila de Reserva ocupa un lugar de la clase. El esquema no tiene fecha ni horario; este cálculo describe los registros actuales, no una ocupación por hora o por mes. Tampoco mide alumnos únicos si una persona tuviera varias reservas.

Ocupación (%) = cantidad de reservas ÷ cupo × 100. Basico: 2 ÷ 4 × 100 = 50 %. Intermedio: 1 ÷ 6 × 100 = aproximadamente 16,67 %. Avanzado: 0 %.

```
SELECT c.IdClase, c.Nombre AS Clase,
       COUNT(r.IdReserva) AS CantidadReservas,
       c.Cupo,
       CAST(100.0 * COUNT(r.IdReserva)
            / NULLIF(c.Cupo, 0)
            AS DECIMAL(7, 2)) AS OcupacionPorcentaje
FROM #ClaseDemo AS c
LEFT JOIN #ReservaDemo AS r
    ON c.IdClase = r.IdClase
GROUP BY c.IdClase, c.Nombre, c.Cupo
ORDER BY c.IdClase ASC;
```

• 100.0 participa como número decimal y evita perder la fracción por una división sólo entre enteros.

• NULLIF(c.Cupo, 0) devuelve el cupo si es distinto de cero; si es cero devuelve NULL, evitando dividir por cero. Un porcentaje NULL requiere revisar la capacidad, no interpretarlo como cero.

• CAST convierte el resultado al tipo elegido. DECIMAL(7, 2) admite siete dígitos en total, dos después del separador decimal; aquí muestra 50.00, 16.67 y 0.00.

• Cupo se agrega al GROUP BY porque participa como columna sin agregación.

Una ocupación superior a 100 % señala más reservas que capacidad según este supuesto. La consulta no impide sobrecupos; sólo los mide.

12 / Lugares libres e informe del club

Lugares libres = cupo de la clase - cantidad de reservas. Debe restarse el conteo de reservas reales, no las filas conservadas artificialmente por un LEFT JOIN.

```
SELECT c.IdClase, c.Nombre AS Clase, c.Cupo,
       COUNT(r.IdReserva) AS CantidadReservas,
       c.Cupo - COUNT(r.IdReserva) AS LugaresLibres
FROM #ClaseDemo AS c
LEFT JOIN #ReservaDemo AS r
    ON c.IdClase = r.IdClase
GROUP BY c.IdClase, c.Nombre, c.Cupo
ORDER BY c.IdClase ASC;
```

Clase	Cupo	Reservas	LugaresLibres
Basico	4	2	2
Intermedio	6	1	5
Avanzado	5	0	5

Basico tiene 4 lugares de capacidad y 2 reservas, así que quedan 2. Intermedio conserva 5 y Avanzado conserva los 5 porque no tiene reservas. Si el resultado fuera negativo, habría más reservas que lugares; no lo ocultamos cambiándolo automáticamente a cero.

Filtrar el informe después de resumir

Para quedarnos con las clases que conservan al menos cuatro lugares, insertamos esta cláusula después de GROUP BY y antes de ORDER BY:

```
HAVING c.Cupo - COUNT(r.IdReserva) >= 4
```

Se conservan Intermedio y Avanzado, con cinco lugares libres cada una. El criterio depende de una resta que incluye un conteo, por eso filtramos grupos con HAVING.

En tu proyecto A05 vas a aplicar esta estructura a ClubPadel y a un umbral diferente. Debés contrastar cada clase con sus reservas; el ejemplo de arriba no es el resultado de tu práctica.

13 / Errores frecuentes y comprobaciones

Problema	Cómo identificarlo y corregirlo
SUM(IdReserva) para contar	Suma etiquetas. Elegí COUNT para cantidad de registros.
COUNT(*) tras LEFT JOIN	La clase vacía aparece con una reserva. Contá r.IdReserva.
INNER JOIN para incluir vacías	Octava desaparece. Conservá Clase a la izquierda con LEFT JOIN.
Sumar Cupo sobre la unión	Capacidad inflada por reservas múltiples. Sumá en Clase.
Falta una columna en GROUP BY	SQL Server indica que la columna no está agregada ni agrupada. Revisá las columnas sin agregación de SELECT.
GROUP BY sólo por Nombre	Puede mezclar clases distintas con igual nombre. Incluí IdClase.
WHERE COUNT(...)	La condición depende del resumen. Pasala a HAVING.
WHERE r.IdReserva IS NULL al contar todas	Se conservan sólo clases vacías. Quitá ese filtro si querés el informe completo.
División entre enteros	El porcentaje pierde la fracción. Usá 100.0 antes de dividir.
IdAlumno comparado con IdClase	El código ejecuta, pero asocia clases incorrectas. Uní r.IdClase con c.IdClase.
GROUP BY usado como orden	Agrupar no ordena. Especificá ORDER BY.

Cómo verificar cada entrega

Revisá qué representa una fila del resultado, la tabla de origen, el total manual y el tratamiento de Octava. En un informe de reservas por clase sin filtros, la suma de los conteos por clase debe coincidir con el conteo de la tabla Reserva, dadas las claves foráneas actuales.

No borres ni insertes reservas para hacer coincidir un resultado. Si algo difiere de la fuente, primero revisamos ON, filtros, agrupación y datos actuales.

14 / Resumen y glosario de repaso

Si necesitás...	Usá...
Cantidad total de filas	COUNT(*) sobre la fuente adecuada
Cantidad de reservas en LEFT JOIN	COUNT(r.IdReserva)
Suma de capacidades	SUM(Cupo) sobre Clase
Un resumen por clase	GROUP BY con IdClase y las columnas mostradas
Filtrar filas antes de agrupar	WHERE
Filtrar según el resumen	HAVING
Ordenar la presentación	ORDER BY

Glosario

Agregación: Cálculo que resume varias filas o valores.

Grupo: Conjunto de filas con iguales valores en las columnas de agrupación.

COUNT: Función que cuenta filas o valores no NULL, según su argumento.

SUM: Función que suma valores numéricos no NULL.

GROUP BY: Cláusula que define los grupos del resumen.

HAVING: Condición aplicada a grupos o agregados.

NULL: Ausencia de valor; no equivale a cero.

Alias: Nombre de presentación de una columna o nombre corto de una tabla.

Capacidad o cupo: Cantidad de lugares que admite una clase en este ejercicio.

Ocupación: Relación porcentual entre reservas registradas y cupo bajo el supuesto indicado.

DECIMAL: Tipo numérico que permite una precisión y una cantidad de decimales definidas.

NULLIF: Función que devuelve NULL si sus dos argumentos son iguales.

15 / Práctica, referencias y continuidad

Cómo vamos a trabajar

Primero estudiamos un concepto con un ejemplo, después recibís una sola consigna. Intentás resolverla en SSMS; revisamos consulta, resultado y motivo de la elección. Ante un error, recibís una pista antes de la solución completa. Los ejemplos resueltos no suman progreso.

A01 y A02 construyen los totales. A03 reúne JOIN, COUNT y GROUP BY. A04 convierte ese resumen en un porcentaje. A05 integra los lugares libres y HAVING en un informe autónomo. El denominador de la barra permanece en cinco.

Referencia al documento original

Cronograma_CSharp_NET.pdf, página 1: estructura de cuatro horas y límite de carga. Página 2, Semana 2, viernes 30 de octubre de 2026: U2 SQL Server - Agregaciones; COUNT, SUM, GROUP BY y HAVING; ocupación, reservas y cupos por clase; cinco KPIs correctos en SQL.

Documentación oficial consultada

Microsoft Learn - COUNT

Microsoft Learn - SUM

Microsoft Learn - GROUP BY

Microsoft Learn - HAVING

Microsoft Learn - NULLIF

Microsoft Learn - CAST y CONVERT

Consultada el 30/10/2026. Explicaciones y ejemplos propios. Cálculos y resultados contrastados en un modelo local; las construcciones específicas de T-SQL se revisaron con la documentación. No se ejecutó código en tu servidor desde este entorno.

Registro al cerrar

Dejaremos: actividades efectivamente completadas, errores corregidos, puntos a reforzar, consulta exacta pendiente si la hubiera, base/herramienta y siguiente punto del plan. Hoy arrancamos con 0 de 5; este PDF no certifica actividades resueltas.

Si seguimos en otra conversación, copiá el cierre y adjuntá el cronograma. Conservá las consultas como Agregaciones_2026-10-30.sql cuando las guardes; el guardado sólo se dará por confirmado si lo verificás.